LAPORAN

WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)

SEMESTER III



BKPM Acara 19 Postman dan API authentication

NAMA:

AZIRA MUTIA GANI

NIM:

E41250290

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Autentikasi merupakan salah satu bagian penting dalam pengembangan aplikasi berbasis web. Autentikasi digunakan untuk memastikan identitas pengguna sebelum pengguna memperoleh akses terhadap layanan tertentu. Pada aplikasi yang menggunakan RESTful API, proses autentikasi dapat dilakukan melalui endpoint yang menerima data pengguna, memeriksa kelengkapan informasi, dan menyimpan identitas pengguna ke dalam basis data.

Salah satu tahap awal penerapan autentikasi adalah registrasi pengguna. Proses registrasi membutuhkan penyimpanan data seperti username, email, dan password. Data tersebut perlu dikelola dengan baik agar tidak terjadi duplikasi username maupun email. Selain itu, password tidak boleh disimpan dalam bentuk teks biasa karena dapat menimbulkan risiko keamanan apabila basis data diakses oleh pihak yang tidak berwenang.

Pada praktikum Acara 19, dilakukan pengembangan RESTful API pada Sistem Informasi Akademik dengan menambahkan fitur registrasi pengguna. Pengembangan meliputi pembuatan tabel `users`, Model `User`, Controller `AuthController`, serta penambahan endpoint `register` pada Router. Password diamankan menggunakan fungsi `password_hash()` dengan algoritma `PASSWORD_DEFAULT`, sedangkan pengujian dilakukan menggunakan Postman.

Melalui praktikum ini, mahasiswa diharapkan memahami proses registrasi pengguna, pengelolaan data menggunakan PHP dan MySQL, serta penerapan keamanan dasar pada API sebelum melanjutkan ke proses login dan autentikasi pada praktikum berikutnya.

1.2 Tujuan Praktikum

1 Memahami konsep dasar registrasi pengguna pada RESTful API.

1 Membuat tabel `users` untuk menyimpan data pengguna.

2 Membuat Model `User` untuk mengelola data pengguna pada basis data.

3 Membuat Controller `AuthController` untuk menangani proses registrasi.

4 Mendaftarkan endpoint `POST /register` pada Router.

5 Menerapkan pengamanan password menggunakan `password_hash()`.
6 Menguji proses registrasi menggunakan Postman.
7 Menerapkan validasi email dan pengaturan environment pada Postman.

BAB 2. LANDASAN TEORI

8.1 RESTful API

RESTful API merupakan antarmuka yang memungkinkan aplikasi client berkomunikasi dengan server menggunakan protokol HTTP. API menerima request, memproses data, kemudian mengirimkan response yang dapat dibaca oleh client.

8.2 Registrasi Pengguna

Registrasi pengguna merupakan proses pembuatan akun baru pada aplikasi. Informasi yang biasanya diperlukan meliputi username, email, dan password.

Proses registrasi perlu memeriksa kelengkapan data dan memastikan informasi yang digunakan memenuhi aturan aplikasi. Kolom username dan email pada tabel pengguna juga perlu memiliki batasan unik agar tidak terjadi penggunaan nilai yang sama oleh beberapa akun.

8.3 Data MySQL

MySQL merupakan sistem manajemen basis data relasional yang digunakan untuk menyimpan dan mengelola data aplikasi. Dalam praktikum ini, MySQL digunakan untuk menyimpan informasi pengguna pada tabel `users`.

Tabel tersebut memiliki kolom `id`, `username`, `email`, `password`, `role`, dan `created_at`. Kolom `id` digunakan sebagai identitas unik pengguna, sedangkan kolom `role` digunakan untuk menentukan peran pengguna dalam aplikasi.

8.4 Model dan Controller

Model merupakan bagian aplikasi yang bertanggung jawab terhadap interaksi dengan basis data. Pada praktikum ini, Model `User` menyediakan method untuk menyimpan pengguna baru dan mencari pengguna berdasarkan username.

Controller bertanggung jawab mengatur alur pemrosesan request. `AuthController` menerima data registrasi, memeriksa kelengkapan input, mengamankan password, kemudian memanggil Model `User` untuk menyimpan data.

Pemisahan Model dan Controller membuat kode program lebih terstruktur serta mempermudah pemeliharaan aplikasi.

8.5 Password Hashing

Password hashing merupakan proses mengubah password menjadi nilai hash agar password asli tidak disimpan secara langsung di dalam basis data. PHP menyediakan fungsi `password_hash()` untuk menghasilkan hash password.

Pada praktikum ini digunakan `PASSWORD_DEFAULT` sebagai algoritma hashing. Ketika pengguna melakukan registrasi, password diubah menjadi hash sebelum disimpan. Pada proses login berikutnya, password yang dimasukkan dapat diperiksa menggunakan fungsi `password_verify()`.

8.6 HTTP Method dan Status Code

HTTP method menentukan jenis operasi yang dilakukan oleh client terhadap server. Pada endpoint registrasi, metode POST digunakan untuk mengirimkan data pengguna baru.

Beberapa status HTTP yang relevan dalam proses registrasi adalah:

Kode	Keterangan
201 Created	Registrasi berhasil dan data baru dibuat.
400 Bad Request	Data yang diperlukan tidak lengkap atau request tidak valid.
409 Conflict	Terjadi konflik, misalnya username atau email sudah digunakan, jika kondisi tersebut ditangani oleh aplikasi.
422 Unprocessable Content	Data tidak memenuhi aturan validasi apabila aplikasi menggunakan status ini.
500 Internal Server Error	Terjadi kesalahan internal pada server.

Penggunaan status HTTP yang sesuai membantu client memahami hasil pemrosesan request.

8.7 Postman

Postman merupakan aplikasi untuk mengirimkan request HTTP dan memeriksa response dari API. Postman mendukung pengaturan metode HTTP, URL, header, body JSON, serta pemeriksaan status code dan response body.

Dalam praktikum ini, Postman digunakan untuk menguji endpoint registrasi dengan data yang valid maupun tidak valid.

BAB 3. PELAKSANAAN PRAKTIKUM

3.1 Hasil Praktikum

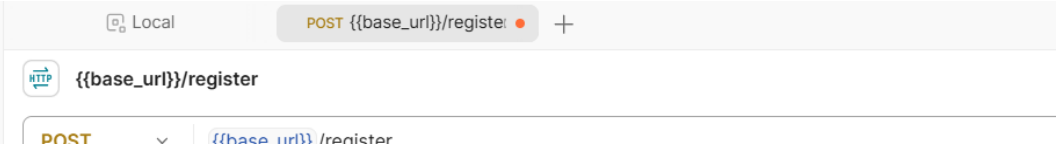
3.1.1 Studi Kasus

Studi kasus pada Acara 19 adalah menambahkan fitur registrasi pengguna ke dalam RESTful API Sistem Informasi Akademik. Pengembangan dilakukan dengan melanjutkan arsitektur MVC yang telah dibuat pada acara sebelumnya.

A. Langkah 1 — Membuat Postman Collection

Langkah pertama adalah membuat Collection pada Postman untuk mengelompokkan request API si-akademik.

Collection diberi nama `API Sistem Informasi Akademik (MVC)`. Selanjutnya, dibuat environment bernama `Local` dengan variabel `base_url` yang mengarah ke alamat dasar aplikasi.



Alamat tersebut merupakan contoh berdasarkan struktur proyek lokal. URL perlu disesuaikan dengan lokasi folder proyek yang digunakan.

Dengan adanya Collection dan environment, request API dapat dikelola secara terpusat. Variabel `base_url` juga mempermudah perubahan alamat server tanpa harus mengubah URL pada setiap request.

B. Langkah 2 — Membuat Tabel `users`

Langkah kedua adalah menambahkan tabel `users` pada basis data yang digunakan oleh Sistem Informasi Akademik. Tabel ini berfungsi untuk menyimpan informasi akun pengguna.

Perintah SQL yang digunakan adalah sebagai berikut:

Run SQL query/queries on database workshop:

```sql
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role ENUM('admin', 'user') DEFAULT 'user',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Keterangan kolom:

1. `id` merupakan identitas unik pengguna yang dibuat secara otomatis.
2. `username` menyimpan nama pengguna dan tidak boleh duplikat.
3. `email` menyimpan alamat email dan tidak boleh duplikat.
4. `password` menyimpan hash password pengguna.
5. `role` menentukan peran pengguna, dengan nilai `admin` atau `user`. Jika tidak ditentukan, nilai awalnya adalah `user`.
6. `created_at` mencatat waktu pembuatan data secara otomatis.

Kolom `password` menggunakan tipe `VARCHAR(255)` agar dapat menampung hash password yang dihasilkan oleh PHP.

C. Langkah 3 — Membuat Model `User`

Langkah ketiga adalah membuat file `User.php` pada folder `app/models/`. Model ini digunakan untuk berinteraksi dengan tabel `users`.

```php
<?php
class User {
    private PDO $db;

    public function __construct() {
        $this->db = Database::getConnection();
    }

    public function create(array $data): string {
        // Tugas Mandiri 1: email harus berformat valid
sebelum disimpan
        if (!filter_var($data['email'],
FILTER_VALIDATE_EMAIL)) {
```

```
            throw new InvalidArgumentException('Format
email tidak valid');
        }
        if ($this->findByUsername($data['username'])) {
            throw new DomainException('Username sudah
digunakan');
        }
        if ($this->findByEmail($data['email'])) {
            throw new DomainException('Email sudah
terdaftar');
        }

        $stmt = $this->db->prepare(
            'INSERT INTO users (username, email,
password) VALUES (:u, :e, :p)'
        );
        $stmt->execute([
            ':u' => $data['username'],
            ':e' => $data['email'],
            ':p' => $data['password'],
        ]);
        return $this->db->lastInsertId();
    }

    public function findByUsername(string $username) {
        $stmt = $this->db->prepare('SELECT * FROM users
WHERE username = :u');
        $stmt->execute([':u' => $username]);
        return $stmt->fetch();
    }

    public function findByEmail(string $email) {
        $stmt = $this->db->prepare('SELECT * FROM users
WHERE email = :e');
        $stmt->execute([':e' => $email]);
        return $stmt->fetch();
    }
}
```

Pada kode tersebut, constructor digunakan untuk mendapatkan koneksi basis data melalui `Database::getConnection()`. Method `create()` menyimpan data pengguna menggunakan prepared statement, sedangkan method `findByUsername()` mencari pengguna berdasarkan username.

Penggunaan prepared statement membantu memisahkan perintah SQL dari nilai input pengguna. Hal ini merupakan salah satu langkah untuk mengurangi risiko SQL injection.

D. Langkah 4 — Membuat Controller `AuthController`

Langkah keempat adalah membuat file `AuthController.php` pada folder `app/controllers/`. Controller ini menangani proses registrasi pengguna.

```
<?php

class AuthController extends BaseController {
    private User $model;

    public function __construct() {
        $this->model = new User();
    }

    // POST /register
    public function register(): void {
        $input = $this->readJson();

        $data = [
            'username' => $this->str($input, 'username'),
            'email'    => $this->str($input, 'email'),
            // password tidak di-trim: spasi di
awal/akhir bisa jadi bagian dari kata sandi
            'password' => isset($input['password']) &&
is_scalar($input['password']) ? (string)
$input['password'] : '',
        ];

        // Tahap 1: field wajib kosong -> HTTP 400
        $missing = [];
        foreach (['username', 'email', 'password'] as
$field) {
            if ($data[$field] === '') {
                $missing[$field] = "Field $field wajib
diisi";
            }
        }
        if ($missing) {
            Response::json(400, false, 'Username, email,
dan password wajib diisi', null, $missing);
        }
```

```
        // Tahap 2: batas panjang sesuai kolom tabel -> HTTP 422
        $errors = [];
        if ($this->len($data['username']) > 50) {
            $errors['username'] = 'Username maksimal 50 karakter';
        }
        if ($this->len($data['email']) > 100) {
            $errors['email'] = 'Email maksimal 100 karakter';
        }
        if ($errors) {
            Response::json(422, false, 'Validasi data gagal', null, $errors);
        }

        // Kata sandi disimpan sebagai hash, tidak pernah plain text
        $data['password'] = password_hash($data['password'], PASSWORD_DEFAULT);

        try {
            $id = $this->model->create($data);
        } catch (InvalidArgumentException $e) {
            Response::json(422, false, 'Validasi data gagal', null, ['email' => $e->getMessage()]);
        } catch (DomainException $e) {
            Response::json(409, false, $e->getMessage());
        }

        // Hash tidak ikut dikirim ke client
        Response::json(201, true, 'Registrasi berhasil', ['id' => (int) $id]);
    }
}
```

Method `register()` membaca data JSON dari request menggunakan `file_get_contents('php://input')`, kemudian mengubahnya menjadi array PHP melalui `json_decode()`.

Selanjutnya, Controller memeriksa apakah username, email, dan password telah diisi. Apabila terdapat data wajib yang kosong, sistem mengirimkan respons HTTP 400.

Jika data wajib tersedia, password diubah menjadi hash menggunakan `password_hash()` sebelum diteruskan ke Model.

Setelah data berhasil disimpan, sistem memberikan respons HTTP 201 beserta ID pengguna yang baru dibuat.

Contoh tersebut menggambarkan alur dasar registrasi. Validasi format email dan penanganan konflik username atau email perlu ditambahkan agar proses registrasi lebih lengkap.

E. Langkah 5 — Mendaftarkan Resource `register` pada Router

Langkah kelima adalah menambahkan resource `register` pada pemetaan Router agar request registrasi dapat diarahkan ke `AuthController`.

```php
<?php
class Router {

    private array $map = [
        'mahasiswa' => 'MahasiswaController',
        'buku'      => 'BukuController',
        'register'  => 'AuthController',
    ];
```

Kemudian, pada method `dispatch()`, ditambahkan penanganan khusus untuk resource `register` sebelum pemrosesan metode HTTP CRUD.

```php
          $controller = new $controllerName();

            // register bukan resource CRUD: hanya
menerima POST
            if ($resource === 'register') {
                if ($_SERVER['REQUEST_METHOD'] !==
'POST') {
                    header('Allow: POST');
                    Response::json(405, false, 'Method
tidak diizinkan');
                }
                $controller->register();
                return;
            }

            switch ($_SERVER['REQUEST_METHOD']) {
                case 'GET':    $controller->index($id);
break;
                case 'POST':   $controller->store();
break;
```

```
                case 'PUT':
                case 'PATCH':  $controller->update($id);
break;
                case 'DELETE': $controller->destroy($id);
break;
```

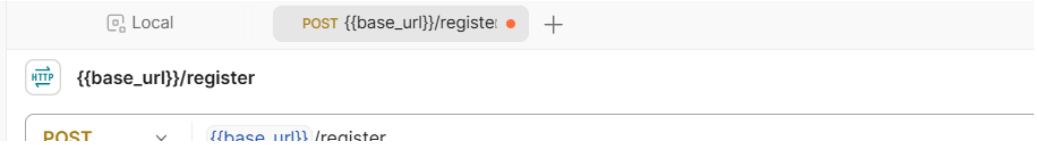
Kode tersebut memastikan bahwa resource `register` diarahkan ke method `register()` pada `AuthController`. Endpoint registrasi hanya menerima metode POST. Jika metode lain digunakan, server mengirimkan respons HTTP 405.

Penanganan khusus diperlukan karena registrasi bukan operasi CRUD biasa yang menggunakan method seperti `index()`, `store()`, `update()`, atau `destroy()`.

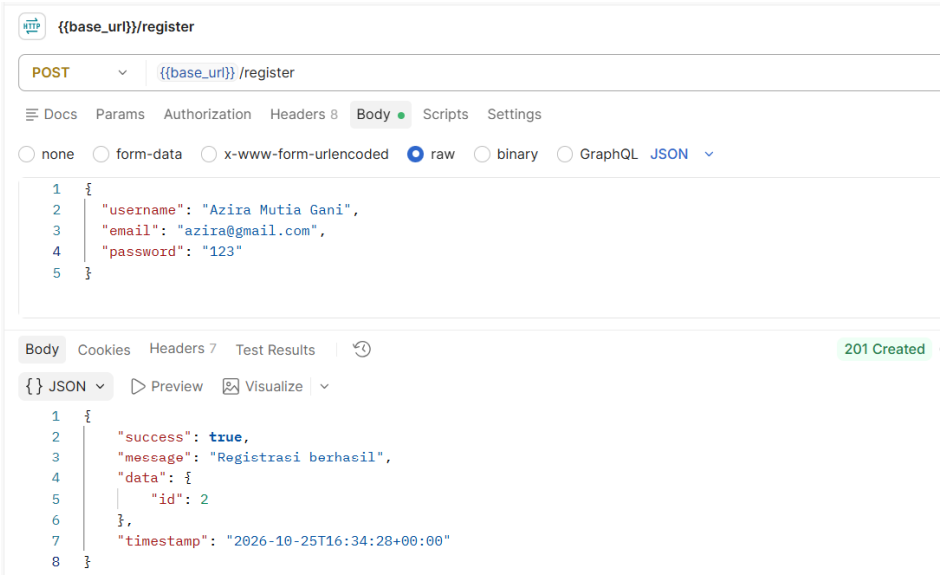
F. Langkah 6 — Menguji Endpoint Registrasi

Langkah keenam adalah melakukan pengujian endpoint registrasi menggunakan Postman.

Pengujian menggunakan metode POST dengan URL:



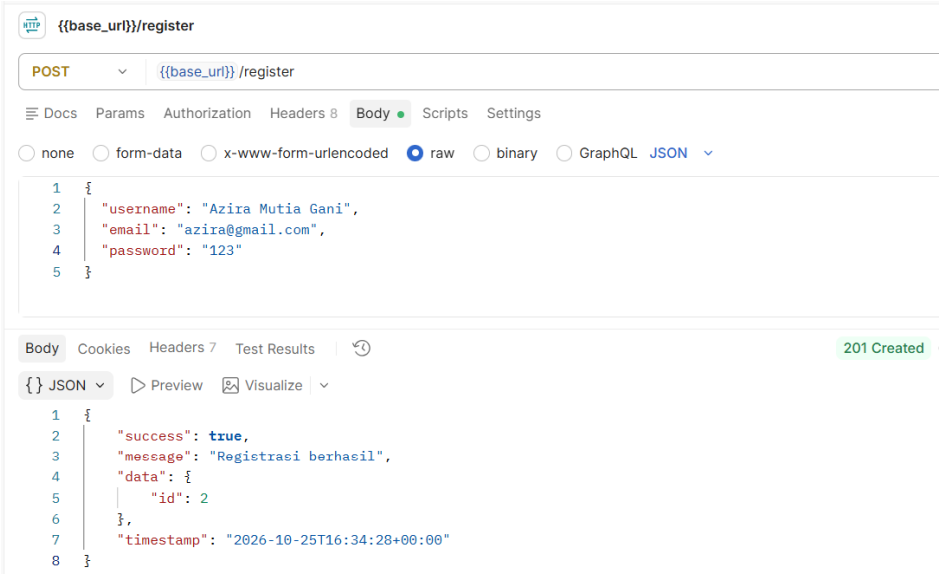
Pada bagian Body, pilih raw dan format JSON.



Data tersebut merupakan data contoh untuk pengujian. Username dan email harus belum digunakan pada tabel `users`.

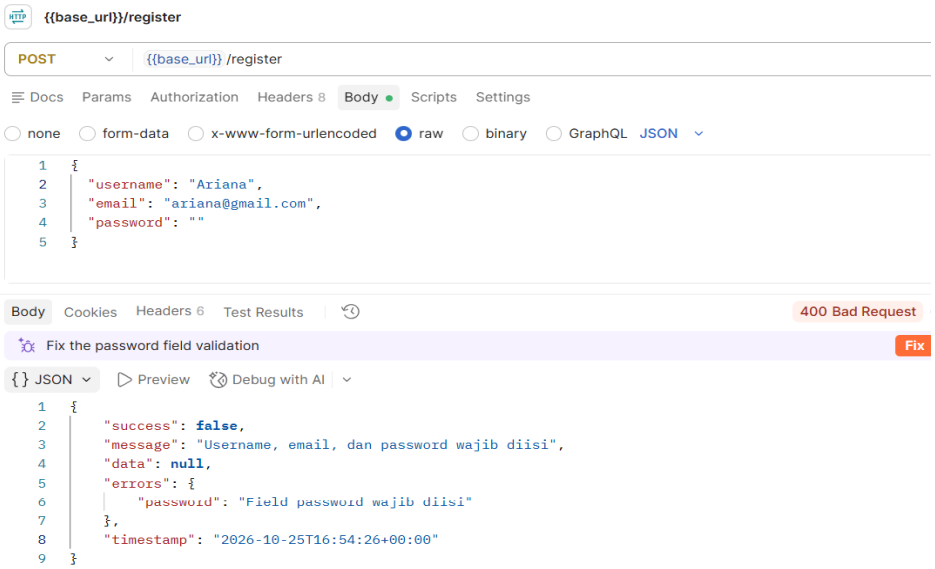
Skenario pengujian yang perlu dilakukan:

1) Registrasi dengan data lengkap dan belum digunakan



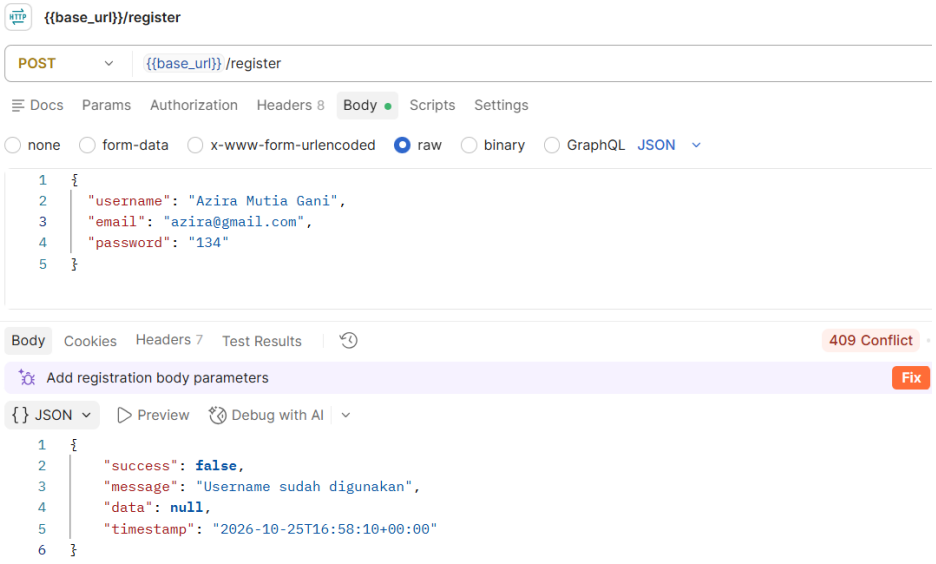
a) Hasil yang diharapkan: HTTP 201 dan pesan registrasi berhasil.

2) Username, email, atau password dikosongkan



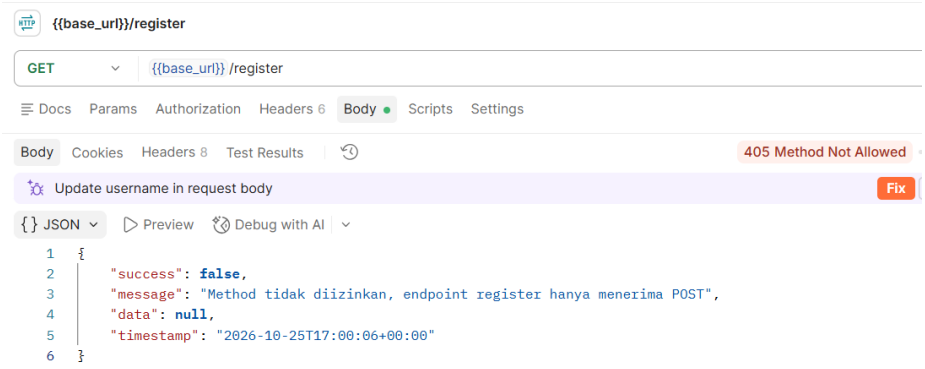
a) Hasil yang diharapkan : HTTP 400 dan pesan field wajib diisi.

3) Username atau email yang sudah digunakan



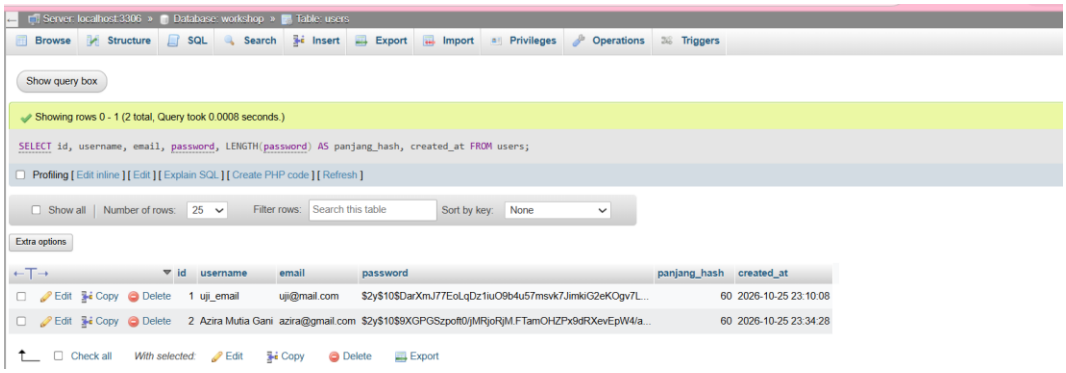
a) Konflik ditangani dengan respons yang sesuai, misalnya HTTP 409.

4) Mengirim request dengan metode selain POST



a) Hasil yang diharapkan: HTTP 405 dan informasi metode yang diizinkan.

5) Memeriksa data hasil registrasi di basis data Data pengguna tersimpan dan password berbentuk hash.



Hasil pengujian aktual dicatat berdasarkan response yang ditampilkan Postman dan pemeriksaan tabel `users`.

3.1.2 Tugas Praktik Mandiri

A. Tugas 1 — Menambahkan Validasi Email

Tugas mandiri pertama adalah menambahkan validasi email agar hanya alamat email dengan format yang benar yang dapat disimpan ke dalam basis data.

Validasi dilakukan menggunakan fungsi `filter_var()` dengan filter `FILTER_VALIDATE_EMAIL`.

```
  public function create(array $data): string {
        // Tugas Mandiri 1: email harus berformat valid
sebelum disimpan
        if (!filter_var($data['email'],
FILTER_VALIDATE_EMAIL)) {
            throw new InvalidArgumentException('Format
email tidak valid');
        }
        if ($this->findByUsername($data['username'])) {
            throw new DomainException('Username sudah
digunakan');
        }
        if ($this->findByEmail($data['email'])) {
            throw new DomainException('Email sudah
terdaftar');
        }
```

Kode tersebut memeriksa format email sebelum data disimpan. Jika format email tidak valid, proses penyimpanan dihentikan dengan melemparkan exception.

Controller perlu menangani exception tersebut dan mengubahnya menjadi respons API yang sesuai, misalnya HTTP 422. Dengan demikian, email yang tidak valid tidak langsung disimpan ke dalam tabel `users`.

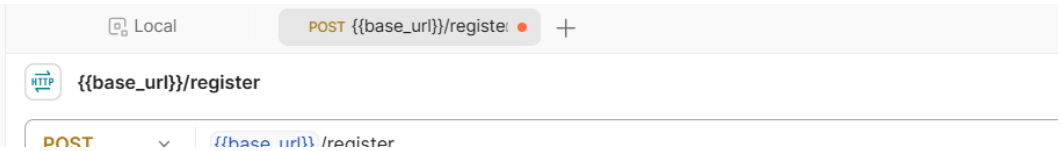
B. Tugas 2 — Membuat Environment Staging pada Postman

Tugas mandiri kedua adalah membuat environment tambahan bernama `Staging` pada Postman. Environment ini digunakan untuk menyimpan konfigurasi alamat server pengujian yang berbeda dari environment lokal.

a) Langkah pengerjaan:

1. Buka Postman dan masuk ke bagian Environments.

2. Buat environment baru bernama `Staging`.
3. Tambahkan variabel `base_url`.
4. Isi nilainya dengan alamat server staging yang benar-benar tersedia.
5. Simpan environment dan gunakan variabel tersebut pada request registrasi.



Nilai `base_url` harus disesuaikan dengan alamat server staging yang tersedia. Jika belum memiliki server staging, environment dapat dibuat terlebih dahulu untuk konfigurasi, tetapi pengujian terhadap server staging belum dapat dinyatakan berhasil.

3.2 Analisis Hasil Praktikum

Praktikum Acara 19 berfokus pada pengembangan fitur registrasi pengguna pada RESTful API. Pengembangan dilakukan dengan menambahkan tabel `users`, Model `User`, Controller `AuthController`, dan pemetaan resource `register` pada Router.

Tabel `users` berperan sebagai tempat penyimpanan identitas pengguna. Batasan `UNIQUE` pada username dan email membantu mencegah duplikasi data, sedangkan kolom `role` menyediakan dasar untuk membedakan peran pengguna pada pengembangan selanjutnya.

Model `User` memisahkan proses interaksi dengan basis data dari alur pemrosesan request. Penggunaan prepared statement membantu mengelola input pengguna dengan lebih aman. Di sisi lain, Controller `AuthController` bertanggung jawab memeriksa kelengkapan data, melakukan hashing password, serta memberikan response setelah proses registrasi.

Penerapan `password_hash()` merupakan bagian penting dalam keamanan akun. Password disimpan dalam bentuk hash sehingga tidak disimpan sebagai teks biasa. Fungsi `findByUsername()` juga menyediakan mekanisme pencarian

pengguna yang dapat digunakan pada pengembangan proses login di acara berikutnya.

Penambahan resource `register` pada Router membuat endpoint registrasi dapat diakses secara terpisah dari endpoint CRUD mahasiswa. Penanganan metode HTTP membantu membatasi proses registrasi agar menggunakan metode POST sesuai rancangan API.

Pada tugas mandiri, validasi email bertujuan mencegah penyimpanan alamat email yang tidak sesuai format. Sementara itu, environment staging pada Postman membantu memisahkan konfigurasi alamat server lokal dan server pengujian.

Keberhasilan praktikum dinilai berdasarkan hasil pengujian aktual, seperti respons registrasi, penanganan input yang tidak lengkap, pencegahan data duplikat, dan penyimpanan password dalam bentuk hash. Jika masih ditemukan kesalahan, pemeriksaan perlu dilakukan pada konfigurasi basis data, pemetaan Router, pemuatan class, serta penanganan exception.

BAB 4. PEMBAHASAN

4.1 Pembahasan

Praktikum Acara 19 membahas pengembangan fitur registrasi pengguna pada RESTful API Sistem Informasi Akademik menggunakan PHP Native dan MySQL. Fitur ini dikembangkan dengan melanjutkan struktur MVC yang telah dibuat pada praktikum sebelumnya.

Pembuatan tabel `users` menjadi tahap awal untuk menyediakan tempat penyimpanan identitas akun. Struktur tabel mencakup username, email, password, role, dan waktu pembuatan akun. Penggunaan batasan unik pada username dan email diperlukan untuk mencegah identitas yang sama digunakan oleh lebih dari satu pengguna.

Model `User` digunakan untuk melakukan operasi terhadap tabel `users`, seperti menyimpan pengguna baru dan mencari pengguna berdasarkan username. Pemisahan tersebut membantu menjaga agar proses pengolahan data tidak bercampur dengan logika penerimaan request.

Controller `AuthController` menangani proses registrasi dengan membaca input JSON, memeriksa kelengkapan data, melakukan hashing password, dan memanggil Model untuk menyimpan pengguna. Penggunaan `password_hash()` dengan `PASSWORD_DEFAULT` membantu melindungi password karena nilai yang disimpan bukan password asli.

Selanjutnya, Router diperbarui dengan menambahkan resource `register`. Penanganan khusus diperlukan karena registrasi merupakan operasi tersendiri dan tidak menggunakan pola CRUD standar. Dengan adanya pemetaan tersebut, request POST ke endpoint `/register` dapat diteruskan kepada method `register()` pada `AuthController`.

Pengujian menggunakan Postman berfungsi untuk memeriksa komunikasi antara client dan server. Melalui pengujian, dapat diperiksa apakah server menerima data registrasi, mengembalikan status HTTP yang sesuai, dan menyimpan data ke basis data. Pengujian terhadap input kosong dan email tidak valid juga diperlukan untuk memastikan bahwa sistem tidak menerima data yang tidak memenuhi ketentuan.

4.2 Analisis Keamanan dan Validasi

Keamanan merupakan aspek penting dalam proses registrasi pengguna. Salah satu penerapan yang dilakukan adalah hashing password menggunakan `password_hash()`. Dengan cara tersebut, password tidak disimpan dalam bentuk teks biasa di basis data.

Selain hashing password, penggunaan prepared statement pada Model membantu mengurangi risiko SQL injection. Pemeriksaan format email juga diperlukan agar data yang disimpan memenuhi ketentuan dasar aplikasi.

Namun, fitur registrasi masih perlu memperhatikan beberapa aspek pengembangan lanjutan. Penanganan username atau email yang sudah digunakan harus menghasilkan respons yang informatif dan tidak menyebabkan aplikasi berhenti dengan kesalahan yang tidak tertangani. Validasi input juga perlu diterapkan secara konsisten pada setiap request yang mengirimkan data pengguna.

Kolom `role` pada tabel `users` menyediakan dasar untuk pengaturan hak akses. Meskipun demikian, pembuatan akun tidak boleh secara bebas memberikan role admin kepada pengguna umum. Pengaturan role admin sebaiknya dilakukan melalui proses yang memiliki otorisasi khusus.

Pengujian pada Postman membantu memeriksa respons API, tetapi tidak menggantikan pemeriksaan langsung terhadap data di basis data. Oleh karena itu, hasil registrasi perlu diverifikasi melalui tabel `users` untuk memastikan bahwa data tersimpan dengan benar dan password telah di-hash.

4.3 Evaluasi Hasil Praktikum

Evaluasi dilakukan dengan membandingkan hasil pengujian dengan perilaku yang diharapkan. Registrasi dengan data valid seharusnya menghasilkan respons HTTP 201 dan identitas pengguna yang baru dibuat. Input wajib yang kosong seharusnya ditolak dengan HTTP 400, sedangkan metode HTTP yang tidak didukung seharusnya ditolak dengan HTTP 405.

Untuk tugas mandiri, email dengan format tidak valid harus ditolak sebelum proses penyimpanan. Environment staging juga perlu diperiksa agar variabel `base_url` mengarah ke server yang sesuai dengan lingkungan pengujian.

Apabila seluruh hasil aktual sesuai dengan skenario yang direncanakan, implementasi dapat dinilai telah memenuhi kebutuhan dasar registrasi. Apabila

terdapat perbedaan, perbaikan perlu dilakukan dan pengujian diulang sebelum hasil akhir didokumentasikan.

BAB 5. PENUTUP

5.1 Kesimpulan

Berdasarkan praktikum Acara 19, dapat disimpulkan bahwa fitur registrasi merupakan salah satu komponen dasar dalam penerapan autentikasi pada RESTful API. Implementasi dilakukan dengan membuat tabel `users`, Model `User`, Controller `AuthController`, dan menambahkan resource `register` pada Router.

Tabel `users` digunakan untuk menyimpan informasi pengguna, sedangkan Model `User` menangani interaksi dengan basis data. Controller `AuthController` mengelola alur registrasi, mulai dari penerimaan input JSON, pemeriksaan kelengkapan data, hingga penyimpanan akun. Penggunaan `password_hash()` membantu menjaga keamanan password dengan menyimpannya dalam bentuk hash.

Pengujian menggunakan Postman dapat membantu memeriksa respons API, penanganan input yang tidak lengkap, serta kesesuaian status HTTP. Tugas mandiri berupa validasi email dan pembuatan environment staging juga mendukung proses pengembangan dan pengujian API secara lebih terstruktur.

Melalui praktikum ini, mahasiswa memperoleh pemahaman mengenai pengelolaan akun pengguna, pemisahan tanggung jawab pada arsitektur MVC, serta penerapan keamanan dasar dalam pengembangan RESTful API.

5.2 Saran

Pengembangan fitur registrasi selanjutnya sebaiknya memperhatikan validasi data, pencegahan username dan email duplikat, serta penanganan kesalahan yang konsisten. Password harus selalu disimpan dalam bentuk hash dan tidak ditampilkan kembali pada response API.

Selain itu, pengujian perlu dilakukan menggunakan berbagai kondisi input, termasuk data lengkap, field kosong, email tidak valid, dan identitas yang telah digunakan. Hasil pengujian sebaiknya disertai bukti screenshot Postman dan pemeriksaan data pada basis data.

Pengembangan berikutnya dapat dilanjutkan dengan fitur login, verifikasi password menggunakan `password_verify()`, pengelolaan token autentikasi, serta pembatasan akses berdasarkan role pengguna. Dengan pengembangan bertahap dan pengujian yang konsisten, sistem dapat menjadi lebih aman dan mudah dipelihara.